Applied Intelligence (2026) 56:483
https://doi.org/10.1007/s10489-026-07520-7

DACD: a dynamic adaptive concept drift learning method for streaming data

Hui Qi[1,2] · Chang Liu[1] · Xiaobo Qi[1,2] · Ying Shi[1,2,3] · Gaoxia Jiang[3]

Received: 6 October 2025 / Accepted: 19 September 2026
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract
Concept drift in streaming data presents a persistent challenge for constructing robust analytical models. Traditional algorithms often exhibit limitations in complex scenarios, performing suboptimally with diverse classification tasks and binary-featured data. This paper introduces the Dynamic Adaptive Concept Drift learning method (DACD) to address these issues. DACD employs a window-based Cumulative Sum (CUSUM) algorithm for drift detection, coupled with a neural network that is updated based on the detected drift type. An adaptive sample processing strategy further enhances model resilience by filtering and augmenting the training data under temporal constraints. The framework selects suitable loss functions optimized with class weights and integrates batch processing and learning rate schedulers to improve training stability. On various benchmark datasets, DACD demonstrates strong performance in accuracy, convergence, and robustness, providing a practical solution for real-world streaming data analysis.

Keywords Streaming data · Concept drift · Adaptive model · Window CUSUM detection · Neural network model

1 Introduction

With the accelerating digital transformation across society, the integration of information and digitalization in various fields has deepened. In numerous scenarios, such as industrial production lines, intelligent transportation, and meteorological warning systems, data are continuously generated in a non-linear, continuous, real-time, and rapid manner. This type of data sequence is known as streaming data. The rational analysis and application of streaming data can enable real-time status monitoring and data prediction [1, 2]. For instance, industrial production lines collect real-time operational status of equipment (e.g., equipment temperature, workshop humidity, product defect rate) to facilitate equipment maintenance, predict failures, and optimize production processes to enhance efficiency. In intelligent transportation systems, in-vehicle systems continuously monitor the vehicle's current location, speed, and surrounding traffic conditions to enable autonomous driving, optimize traffic allocation, and alleviate congestion. Meteorological stations in various locations capture real-time local data, including temperature, humidity, atmospheric pressure, and pollutant concentrations, for weather forecasting and severe weather alerts. In streaming data, the underlying data distribution is often non-linear and tends to evolve over time, a phenomenon known as concept drift [3].

Concept drift is typically classified based on the rate of change in the data distribution into four categories: abrupt, gradual, incremental, and recurring drift [4]. Abrupt drift refers to a significant change in the data distribution over a short period. Gradual drift signifies a slower, less pronounced change in the distribution over time. Incremental

✉ Gaoxia Jiang
jianggaoxia@sxu.edu.cn

Hui Qi
qihui@tynu.edu.cn

Chang Liu
aeolian20071@foxmail.com

Xiaobo Qi
xbqi@tynu.edu.cn

Ying Shi
sy@tynu.edu.cn

[1] School of Computer Science and Technology, Taiyuan Normal University, Jinzhong 030619, Shanxi, China

[2] Shanxi Provincial Key Laboratory of Intelligent Optimization Computation and Blockchain Technology, Jinzhong 030619, Shanxi, China

[3] School of Computer and Information Technology, Shanxi University, Taiyuan 030006, Shanxi, China

Published online: 09 October 2026

drift involves a progressive, non-linear evolution of the data distribution. Recurring drift describes a periodic change in the data distribution over time.

Traditional concept drift classification algorithms are often limited to detecting and distinguishing between different types of concept drift phenomena. However, they typically fail to consider other intrinsic characteristics of the input data, such as the distinction between binary and multi-class classification problems in different scenarios, or data involving binary features like circuit on/off states. In binary classification problems, the model's objective is clear and more amenable to linear separation. In contrast, multi-class problems are more prevalent in real-world scenarios, featuring more complex data distributions that necessitate more sophisticated model architectures to handle the occurrence of concept drift. A simplistic application of general-purpose classification methods may lead to suboptimal performance on real-world datasets. Furthermore, streaming data with binary features often exhibit non-linear changes during concept drift, making them more challenging to adapt to and capture. An additional limitation of traditional machine learning algorithms for concept drift is their computational inefficiency, which makes it difficult to process large volumes of streaming data quickly and effectively.

To better address the challenges of concept drift in the aforementioned diverse scenarios, this paper proposes a dynamic adaptive concept drift learning method for streaming data (DACD). This method employs a window-based cumulative sum approach for concept drift detection and adaptively adjusts the corresponding model and sample selection strategy based on the type and severity of the detected drift. The DACD method utilizes a neural network to process high-dimensional data, incorporates sample selection and augmentation techniques to mitigate class imbalance and insufficient learning, and uses standard optimization strategies to support stable neural-network training. The main contributions of this paper are as follows:

1. A dynamic adaptive concept drift learning framework for block-based data streams is proposed, in which window CUSUM-based drift detection, drift-type-aware sample processing, and neural-network updating are integrated into a sequential chunk-level learning process.
2. An adaptive window CUSUM-based drift detection mechanism is constructed to characterize chunk-level distribution changes through the drift boundary parameter and the drift severity coefficient, thereby distinguishing abrupt drift from gradual drift.
3. A drift-type-aware historical sample filtering and training sample augmentation strategy is designed to dynamically regulate the participation of historical samples according to drift type and drift severity, improving the adaptability and stability of the model under different concept drift scenarios.

2 Related work

When processing streaming data from diverse scenarios, the intertwined problems of concept drift and class imbalance significantly increase the difficulty of data analysis and model construction. In most real-world streaming data, class imbalance is a prevalent issue. Once a change occurs in the minority class, the number of minority class samples is highly likely to decrease further. The stable distribution of the majority class can mask these subtle changes, thereby preventing the model from capturing critical features in a timely manner and causing it to overlook the transition process between the majority and minority classes. Furthermore, in complex streaming data, model performance is affected not only by concept drift but also by data complexity, such as multi-faceted features, multi-class settings, and data noise, posing immense challenges to the model's detection accuracy and spatio-temporal complexity.

To address these potential issues, learning methods for concept drift typically employ either block-based or online learning approaches for drift detection and model training [5].

Block-based methods are a significant training strategy in the field of streaming data classification. In this approach, streaming data instances are segmented into data chunks of equal size, which then serve as the basic units for constructing and training classifiers. SERA [6] is a block-based method designed to handle class imbalance. It can selectively add minority class samples from historical data chunks to the current block and proportionally assign sampling probabilities to majority and minority class samples, effectively mitigating class imbalance under concept drift. The SEA method [7] divides streaming data into consistently sized blocks, builds a separate classifier on each consecutive training block, and then uses weighted voting to determine the final classification. Its classification accuracy is comparable to that of a single decision tree, and it can adapt quickly to concept drift. DWSE [8] increases the number of minority class samples in the current block by resampling them from historical blocks, incorporating information from historical data to build the classifier. This approach effectively reduces the impact of concept drift and improves classification performance for the minority class. However, because block-based methods [9, 10] need to store multiple data chunks to build, train, and evaluate classifiers, they consume substantial memory. Furthermore, these methods struggle to react to concept drifts occurring within a single block, making them unable to finely capture intra-block variations and causing their performance to be significantly impacted by frequent and subtle abrupt drifts.

The core principle of online learning-based methods is the real-time processing of data. As data continuously flows, each sample obtained from the stream is processed only once, without the need for secondary storage or reprocessing. Compared to block-based methods, they are generally faster and more accurate, which has led to their widespread application. OGUEIL [11] utilizes an online decision tree and a G-mean-based online weighting mechanism, combined with oversampling and an adaptive sliding window technique, to incrementally update base classifiers and their weights as data arrives. AC_OE [12] constructs local learners through an online ensemble, performing local predictions on training samples within a data chunk to dynamically adjust learner weights. It then integrates this with an incremental learning strategy to build a global incremental learner that is updated as new samples arrive. Targeted EL [13] employs different sample weighting strategies for different types of drift. It constructs a difference matrix and, based on the drift type, adaptively extracts a sub-matrix of appropriate size with the maximum difference sum to select suitable base learners for the ensemble. A significant drawback of online learning methods [14–16] is their susceptibility to noisy data. Since each sample is processed only once, the model can be misled by contaminated data, and this error is difficult to correct with subsequent samples, leading to a predicament of overfitting to data noise, which severely impacts classification accuracy.

The DACD method proposed in this paper processes streaming data in a block-based manner. Each data chunk is handled sequentially after it arrives, and the model is updated using the current chunk together with previously observed chunks, without accessing future samples. To tackle the class imbalance problem, DACD employs a multi-faceted strategy that includes remapping labels and calculating class weights during data preprocessing, adjusting the loss value based on these weights during model training, and determining class sampling priorities and augmenting the training set with historical samples through an adaptive sample processing strategy. These measures enhance the model's performance in complex data scenarios. To address issues such as multi-faceted features, multi-class settings, and data noise, DACD utilizes standardization during data preprocessing; employs a window-based cumulative sum for drift detection and adaptive parameter adjustment; leverages features of the neural network model such as batch normalization, an optional embedding layer, and appropriate loss functions; and combines these with adaptive sample selection and augmentation.

Accordingly, these limitations motivate DACD to combine block-level processing, drift-type-aware historical sample reuse, and neural network retraining within a temporally constrained workflow. The following sections describe these components and then evaluate them empirically across multiple drift scenarios.

3 DACD

This paper proposes a method specifically designed to address the phenomenon of concept drift in streaming data. The proposed method, DACD, is composed of four main components: data preprocessing, a window-based cumulative sum (CUSUM) drift detection method, neural network model training, and adaptive sample processing. The core idea is to monitor dynamic changes in the data distribution at the chunk level during the continuous inflow of data, identify drift phenomena after each chunk is received, and dynamically adjust the model according to the data characteristics and drift type. Concurrently, the data undergoes corresponding preprocessing, sample selection, and augmentation to ensure that the model can maintain continuous and accurate classification performance. The architecture of the DACD method is illustrated in Fig. 1.

In the data preprocessing stage, standardization is applied based on the characteristics of the streaming data to provide a solid foundation for subsequent model training.

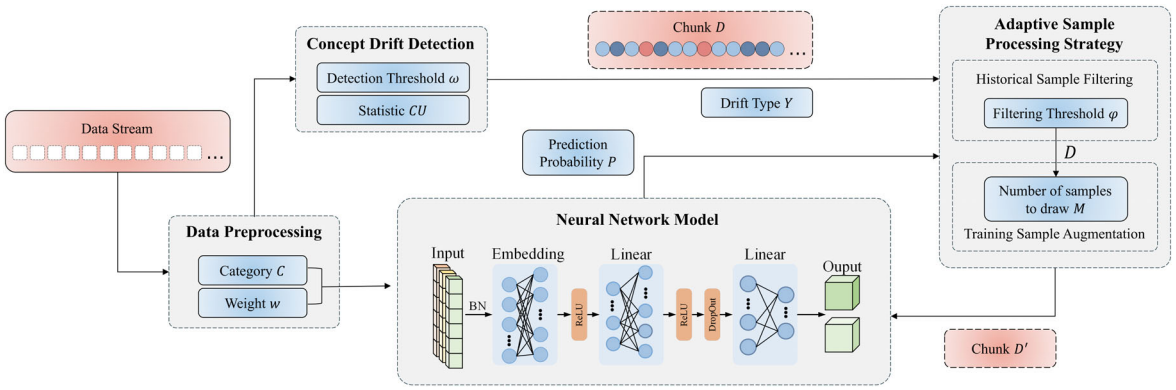


Fig. 1 A holistic pipeline of the DACD method for chunk-wise concept drift handling with a focus on adaptive sample processing and model retraining

Simultaneously, labels are remapped and class weights are calculated to address potential class imbalance issues that may arise. For concept drift detection, the window-based CUSUM detection method determines the drift point, scope, and type by calculating a statistic based on a reference distribution and standard deviation. During this process, key parameters are adaptively adjusted to enhance detection efficiency. DACD utilizes a neural network model as its core classifier, selecting appropriate loss functions for different classification problems and incorporating a learning rate optimizer and scheduler to make the training process more efficient. The adaptive sample processing strategy of DACD selects from historical samples and augments the training set based on the results from the CUSUM detection component, which effectively enhances the model's adaptability across different concept drift scenarios.

3.1 Data preprocessing

Scaling the dataset features to a distribution with a mean of 0 and a standard deviation of 1 is beneficial for model training. Therefore, a data standardization operation is employed in the data preprocessing stage:

$$x_{scaled} = \frac{x - \mu}{\sigma} \tag{1}$$

where x is the original feature value, μ is the mean of the feature, and σ is the standard deviation of the feature. Additionally, labels are remapped, and class weights are calculated to address potential class imbalance:

$$w_i = \frac{N}{K \times n_i} \tag{2}$$

where w_i is the weight for class i, n_i is the number of samples in class i, N is the total number of samples, and K is the total number of classes. This preprocessed data provides a suitable input for the subsequent drift detection and model training phases.

3.2 Concept drift detection

The standard Cumulative Sum (CUSUM) algorithm detects changes by accumulating deviations from a reference value and flagging a drift when this sum surpasses a predefined threshold [17]. However, traditional CUSUM is highly sensitive to its parameter settings and often struggles to effectively handle abrupt drifts or drifts that occur over an extended period. To address these limitations, we propose a window-based, adaptive CUSUM method tailored for multivariate data streams.

The procedure for our proposed method is illustrated in Fig. 2. The process begins by establishing a reference distribution from a window of recent, stable data chunks. At chunk D_t, this window contains only chunks that have been completed before D_t; samples from the current chunk and future chunks are not used to form the reference distribution. Features from these chunks are vertically concatenated to form a reference dataset. The mean μ_{ref} and standard deviation σ_{ref} of this reference dataset are then computed as follows:

$$\mu_{ref} = \frac{1}{N}\sum_{i=1}^{N} x_i \tag{3}$$

$$\sigma_{ref} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu_{ref})^2} \tag{4}$$

where N is the number of samples in the reference dataset.

For each incoming data sample $x_{i,j}$ (where i denotes the sample index and j denotes the feature index) in the current data chunk, we first compute its standardized difference $d_{i,j}$ relative to the reference distribution:

$$d_{i,j} = \frac{x_{i,j} - \mu_{ref,j}}{\sigma_{ref,j}} \tag{5}$$

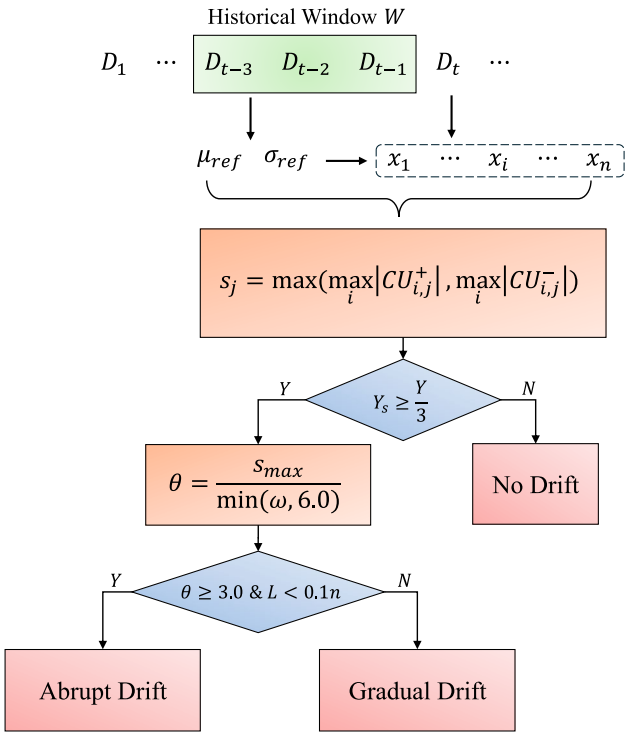


Fig. 2 The concept drift detection procedure, where s_{max} is the maximum value of drift score s_j

Subsequently, a two-sided CUSUM statistic, $CU^{+}_{i,j}$ and $CU^{-}_{i,j}$, is updated for each feature:

$$\begin{cases} CU^{+}_{i,j} = \max(0, CU^{+}_{i-1,j} + d_{i,j} - \delta) \\ CU^{-}_{i,j} = \min(0, CU^{-}_{i-1,j} + d_{i,j} + \delta) \end{cases} \tag{6}$$

An adaptive component of the method is the adjustment of the drift boundary δ_t. This parameter provides a baseline for drift detection and is dynamically tuned using only the most recently detected drift type. The initial value is $\delta_0 = 0.3$. After an abrupt drift, the next detection window uses $\delta_{t+1} = 0.1$ to increase sensitivity; after a gradual drift, the boundary is restored to $\delta_{t+1} = 0.3$ to reduce false alarms caused by slow fluctuations. If no drift is detected, $\delta_{t+1} = \delta_t$:

$$\delta_{t+1} = \begin{cases} 0.1, & \text{if the latest detected drift is abrupt,} \\ 0.3, & \text{if the latest detected drift is gradual,} \\ \delta_t, & \text{if no drift is detected.} \end{cases} \tag{7}$$

After processing the chunk, the drift score s_j for the j-th feature is determined by the maximum absolute CUSUM value observed:

$$s_j = \max(\max_i |CU^{+}_{i,j}|, \max_i |CU^{-}_{i,j}|) \tag{8}$$

A concept drift event at the chunk level is confirmed if the number of features (Y_s) whose drift scores s_j exceed a drift threshold is greater than or equal to one-third of the total feature count. Let $\mathcal{J}_d = \{j \mid s_j > \omega\}$ denote the set of drifted features, and let $s_{max} = \max_j s_j$ denote the maximum drift score among all features. After this chunk-level drift condition is satisfied, the drift severity coefficient is calculated as:

$$\theta = \frac{s_{max}}{\min(\omega, 6.0)}. \tag{9}$$

The coefficient θ is not used as an independent drift detector; rather, it measures the severity of an already confirmed drift event. The denominator normalizes the strongest feature-level drift score by the detection threshold, while the constant 6.0 acts as an upper bound for this normalization factor when a larger ω is used. For a confirmed drift event, at least one feature satisfies $s_j > \omega$, and therefore $s_{max} > \omega$. Consequently, $\theta > 1$ indicates that the strongest feature-level drift score exceeds the normalized detection threshold. When the chunk-level drift condition is not satisfied, no drift event is reported and θ is set to 0; thus, the value range of θ is $\{0\} \cup (1, +\infty)$.

If a drift is detected, the first and last potential drift points within the chunk are designated as the drift start (x_s) and end (x_e) points, respectively, defining a drift range of $L = x_e - x_s + 1$.

Finally, the drift is classified according to the drift range L and the drift severity coefficient θ . The severity cutoff is set to $\theta = 3.0$. For a confirmed drift event, $\theta \geq 3.0$ indicates that the strongest feature-level change is at least three times the normalized detection threshold, while $L < 0.1S$ indicates that the change is concentrated within a short segment of the current chunk. Therefore, an event satisfying both $\theta \geq 3.0$ and $L < 0.1S$ is categorized as an abrupt drift; otherwise, it is classified as a gradual drift. This combination prevents long-duration gradual changes from being labeled as abrupt solely because of a large drift score, and the resulting drift type informs the adaptive adjustment of δ for subsequent detection windows.

3.3 Neural network model and training

The DACD method employs a neural network model as its core classifier. The model is first set to training mode, and the labels are remapped. A corresponding loss function is selected based on the type of classification problem, taking class weights into account. Subsequently, an optimizer and a learning rate scheduler are utilized to update model parameters and dynamically adjust the learning rate according to the gradients. After enabling mixed-precision training, for each batch, the gradients in the optimizer are first cleared. The forward propagation is then performed under mixed-precision training to compute the model's output and the corresponding loss. Next, backpropagation is executed to calculate the gradients and update the model parameters. The loss is accumulated, and the learning rate is adjusted based on the total accumulated loss. This iterative process continuously enhances the classification accuracy. The architecture of the neural network is depicted in Fig. 3.

3.3.1 Forward propagation process

The neural network model is composed of a batch normalization layer, an embedding layer (for datasets with binary features), and a main classifier layer.

1. **Batch Normalization Layer**
   A batch normalization operation is applied to the input features to mitigate the problems of vanishing or exploding gradients. Let the input feature be x_{ij} (the j-th feature of the i-th sample). The mean μ_j and variance σ_j^2 of x_{ij} within the current batch are calculated, where n_{batch} is the number of samples in the current batch:

$$\mu_j = \frac{1}{n_{batch}} \sum_{i=1}^{n_{batch}} x_{ij} \tag{10}$$

$$\sigma_j^2 = \frac{1}{n_{batch}} \sum_{i=1}^{n_{batch}} (x_{ij} - \mu_j)^2 \tag{11}$$

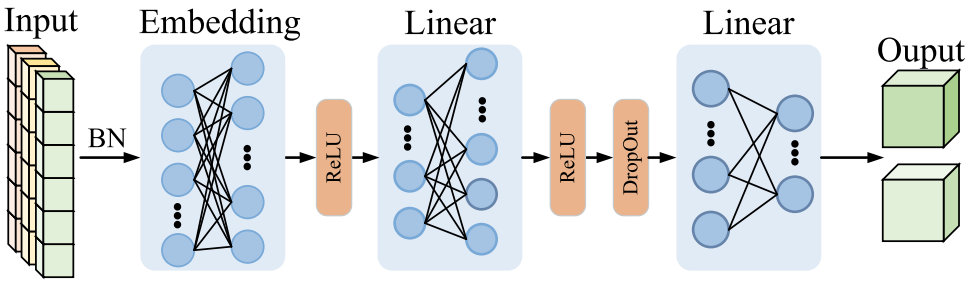


Fig. 3 Neural network model architecture

The feature x_{ij} is then normalized (ϵ is a small constant):

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} \tag{12}$$

After normalization,the output y_{ij} is scaled and shifted, where γ_j is a weight parameter that determines the influenceof the input x_{ij}, and β_j; is a bias parameter used to adjust the baseline level of the output:

$$y_{ij} = \gamma_j \hat{x}_{ij} + \beta_j \tag{13}$$

2. **Embedding Layer**
An optional embedding layer is designed for high-dimensional datasets that contain binary features. This layer transforms the discrete features of a sample into continuous vectors, adding extra learnable parameters to the model. This allows the model to learn more complex feature relationships and capture underlying patterns in the data. The optional embedding layer is essentially a linear transformation layer:

$$y = Wx + b \tag{14}$$

where y is the output vector of the embedding layer, and x is the output vector from the batch normalization layer. W is the weight matrix used to map the low-dimensional input vector to a high-dimensional space, and b is the bias vector.

3. **Main Classifier Layer**
The main classifier consists of four sub-layers. Let x be the input features, W_1 and W_2 be the weight matrices, b_1 and b_2 be the bias vectors, and $ReLU(z) = max(0, z)$ be the activation function:

$$h(x) = ReLU(W_1 x + b_1) \cdot W_2 + b_2 \tag{15}$$

The first fully connected layer maps the input features to a 128-dimensional hidden space. The output of this layer is the sum of the product of the weight matrix W_1 and the output x from the previous layer, and the bias vector b_1. Each element of the output vector y_1 from the fully connected layer is then activated by the activation function, yielding $y_2 = ReLU(y_1) = max(0, y_1)$.
To prevent the neural network from overly relying on specific neurons, which can lead to overfitting during model training, a Dropout layer is employed to set certain neurons to zero. In the second fully connected layer, the 128-dimensional hidden space is mapped to the output dimension. The output dimension is 1 for binary classification problems and the number of classes for multi-class problems. The output of this layer, $h(x) = y_4 = W_2 \cdot y_3 + b_2$, is used for subsequent classification decisions.

3.3.2 Dynamic class-weighted loss function

DACD uses a dynamic class-weighted loss mechanism to alleviate class imbalance in streaming data. Class-weighted losses are standard tools; in DACD, this component incorporates chunk-level class statistics into drift-triggered model updates, so that minority classes retain sufficient influence during retraining.

For binary classification tasks, DACD uses a Class-Weighted Binary Cross-Entropy (CBCE) loss. This function enhances the standard binary cross-entropy by integrating the class weight w_{y_i}, calculated as per (2), for each sample. This modification increases the loss contribution from minority class samples and mitigates the risk of being overshadowed by the majority class. The function is formulated as:

$$L_{\text{CBCE}}(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^{N} w_{y_i} \left[y_i \cdot \log(\sigma(\hat{y}_i)) + (1 - y_i) \cdot \log(1 - \sigma(\hat{y}_i)) \right] \tag{16}$$

where y_i is the true label of sample i, $\hat{y}_i$ is the model's raw output, $\sigma(\cdot)$ is the Sigmoid function [18], and w_{y_i} is the pre-calculated weight for the class corresponding to the true label y_i.

Analogously, for multi-class classification problems, DACD utilizes a Class-Weighted Cross-Entropy (CCE) loss. This function directly embeds the class weights into the cross-entropy calculation, increasing the penalty for misclassifying

minority class samples and reducing bias toward dominant classes. The formulation is as follows:

$$L_{\mathrm{CCE}}(y,\hat{y}) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} w_k \cdot y_{i,k} \cdot \log(P(y=k|x_i)) \quad (17)$$

where w_k represents the weight for class k, and $y_{i,k}$ is an indicator that is 1 if the true class of sample i is k, and 0 otherwise. By adjusting the learning focus based on class distribution, this mechanism helps reduce the impact of class imbalance.

3.3.3 Convergence analysis with dynamic weighting

The convergence analysis of DACD follows from the combination of the AdamW optimizer [19, 20] and the boundedness of the dynamic class-weighted loss function. A fundamental prerequisite for the convergence analysis of gradient-based optimizers is that the gradient of the objective function satisfies the Lipschitz continuity condition. Given that the class weights are bounded ($0 < w_i \leq W$, where W is a constant), the Lipschitz constant of the weighted loss function's gradient, ∇L_ω, remains bounded. This is demonstrated by the following continuous inequality, assuming the gradient of the unweighted loss, ∇L, is $\mathcal{L}$-Lipschitz continuous:

$$\begin{aligned}
&\|\nabla L_\omega(\Theta_1) - \nabla L_\omega(\Theta_2)\| \\
&= \left\| \sum_i w_i \left(\nabla L_i(\Theta_1) - \nabla L_i(\Theta_2)\right) \right\| \\
&\leq \sum_i w_i \|\nabla L_i(\Theta_1) - \nabla L_i(\Theta_2)\| \\
&\leq W \sum_i \|\nabla L_i(\Theta_1) - \nabla L_i(\Theta_2)\| \\
&= W \cdot \|\nabla L(\Theta_1) - \nabla L(\Theta_2)\| \\
&\leq (W\mathcal{L})\|\Theta_1 - \Theta_2\|.
\end{aligned} \quad (18)$$

This derivation confirms that the gradient of the class-weighted loss remains Lipschitz continuous, with an upper-bound constant $W\mathcal{L}$, thus satisfying a core condition for the convergence guarantees of the AdamW optimizer.

In practice, the ReduceLROnPlateau scheduler works with the optimizer by dynamically decaying the learning rate whenever the loss value ceases to decrease, facilitating finer-grained adjustments as the parameters approach an optimum.

Consequently, as the number of iterations $t \to \infty$, the gradient norm $\|\nabla L_\omega\| \to 0$, which indicates that the parameter sequence converges to a stationary point of the objective function. This analysis is consistent with the experimental results presented in Section 4.4.4 concerning the recovery and stabilization speed of the DACD method .

3.4 Adaptive sample processing strategy

The adaptive sample processing strategy, which is based on the drift detection results from the data chunk, is primarily divided into two parts: historical sample filtering and training sample augmentation. This strategy dynamically adjusts the samples used for training the model by selecting reliable samples from the historical dataset and combining them with the current data chunk according to the drift type. The process is illustrated in Fig. 4.

3.4.1 Historical sample filtering

When concept drift occurs, historical samples often fail to fully conform to the current data distribution. However, these historical samples should not be entirely discarded. To remove samples that the model predicts with uncertainty and to make the historical samples more aligned with the current data distribution, a sample filtering process is applied to the historical dataset. The historical dataset is constructed from previously completed data chunks, and the number of historical data chunks is limited to ensure model accuracy.

To avoid overfitting and excessive memory overhead from using too many historical samples in model training, when the total number of samples in the historical data chunks becomes too large, the samples are downsampled. A certain number of the most recent high-confidence sample sets are selected. This focuses the model on recent data characteristics and changing trends. The model undergoes multiple training rounds to obtain a low-confidence sample filter F, which is based on the current data distribution and used for subsequent sample filtering.

Predictions are made using a batch processing approach to improve computational efficiency. The trained neural network model is used for prediction, and the prediction probabilities are calculated using the Sigmoid function or the SoftMax function, depending on the type of classification problem. The methods for calculating prediction probabilities are as follows:

1. **Binary Classification Probability Calculation**
   The Sigmoid saturation function has upper and lower bounds of 1 and 0, respectively. This can be interpreted as the probability of the sample belonging to the positive class. The prediction probability is calculated as:

$$P(y=1|x) = \frac{1}{1+e^{-h(x)}} \quad (19)$$

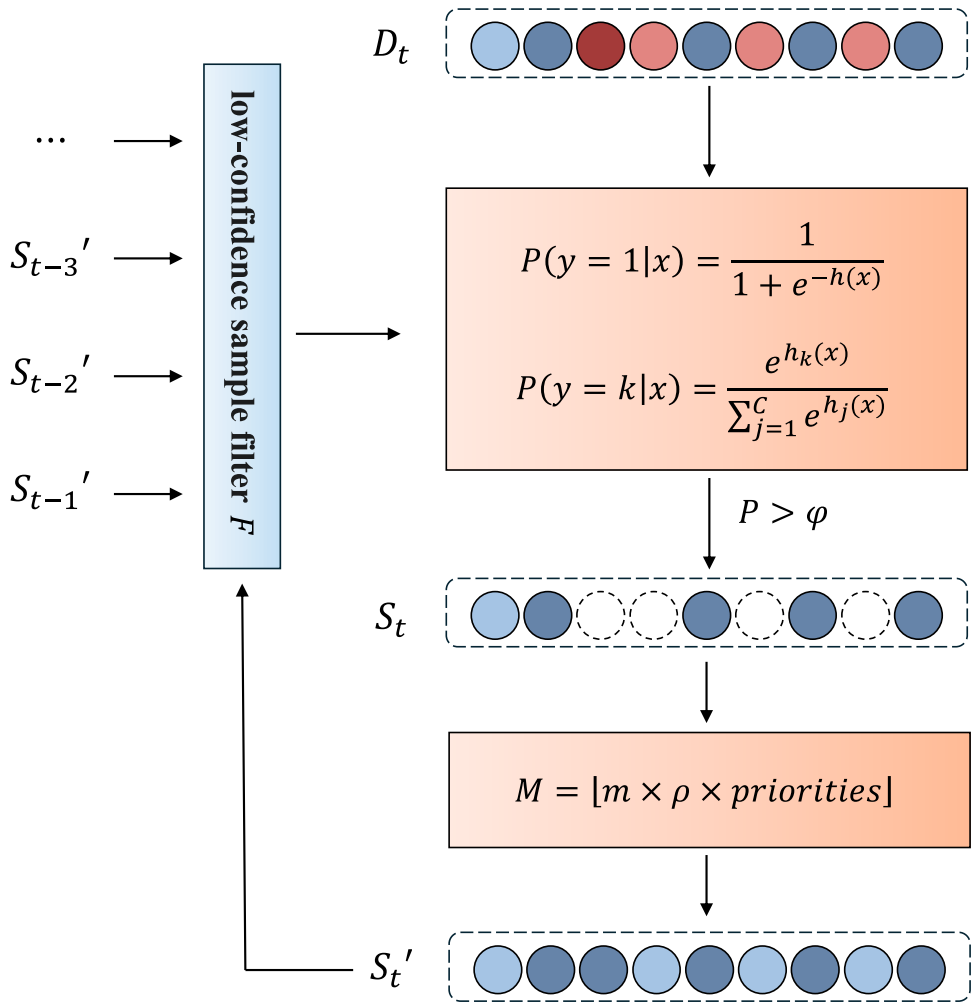


Fig. 4 The adaptive sample processing procedure

where $P(y = 1|x)$ is the probability that sample x belongs to the positive class, and $h(x)$ is the model's output for the positive class.

2. **Multi-Class Classification Probability Calculation**
For multi-class problems, the SoftMax function is used, with the formula as follows:

$$P(y = k|x) = \frac{e^{h_k(x)}}{\sum_{j=1}^{c} e^{h_j(x)}} \tag{20}$$

where $P(y = k|x)$ is the probability that sample x belongs to the k-th class, $h_j(x)$ is the model's output for the k-th class, and C is the total number of classes.
A confidence filtering threshold is calculated based on the drift type and severity to better adapt to varying degrees of concept drift. Since φ is used as a probability threshold, it is clipped to the interval [0, 1] to avoid infeasible values when θ becomes large:

$$\varphi = \begin{cases} \min(1, 0.6 + 0.1 \times \theta) & \text{Abrupt} \\ \min(1, 0.7 + 0.2 \times \theta) & \text{Gradual} \end{cases} \tag{21}$$

When the predicted probability of a candidate historical sample is greater than or equal to the historical data filtering threshold, the high-confidence historical sample set S_t is retained for subsequent processing. The subsequent sampling ratio ρ further controls how many retained historical samples are used for model training under abrupt and gradual drifts.

3.4.2 Training sample augmentation

Samples within the current preprocessed data chunk D_t' may be insufficient for certain classes due to concept drift or other reasons, leading to inadequate model training. To address this issue, D_t' is augmented with the filtered historical sample set S_t from the historical dataset, thereby improving the model's adaptability to different classes of data. The historical dataset used in this step is composed only of data chunks observed before the current update; later data chunks or future stream samples are not used, so the validity of the experimental results is maintained.

To address the potential class imbalance issue within D'_t, it is necessary to determine the sampling priority for each class before augmentation. Let m_a be the number of samples of class a in D'_t, and n be the total number of samples in D'_t. The sampling priority for class a, $priorities_a$, is:

$$priorities_a = 1.0 - \frac{m_a}{n} \tag{22}$$

The number of samples to be drawn, M_a, is calculated based on m_a, the sampling ratio, and the sampling priority:

$$M_a = [m_a \times \rho \times priorities_a] \tag{23}$$

For abrupt drift, the data distribution undergoes significant changes in a short period, rendering most historical samples unsuitable for the current distribution. The sampling ratio is appropriately reduced to lessen their interference with model training, allowing the model to focus more quickly on the current data distribution. For gradual drift, where the data distribution changes more slowly, historical samples share a stronger similarity with the current distribution. Increasing the sampling ratio allows the model to adapt more smoothly to the changes in data distribution.

Historical samples for class a, denoted as $S_t(a)$, are collected from S_t. From these, M_a samples are randomly drawn (if the number of samples in $S_t(a)$ is insufficient, all samples of that class are drawn).

The historical samples drawn for all classes are then vertically stacked with D'_t. The resulting augmented training set, S'_t, provides more sufficient and representative samples for model training. This enhances the model's generalization ability and accuracy when handling data affected by concept drift.

The overall process of the DACD method is presented in Algorithm 1, and a complexity analysis is provided in Appendix A.

Algorithm 1 DACD Algorithm.

Require: Data stream SD, historical window size W_h , threshold ω, initial boundary δ_0, ratios ρ_{abrupt} and ρ_{gradual}, rate α.
Ensure: Trained model M.

```
M ← InitializeNetwork()
W_hist ← ∅
δ ← δ_0
for each data chunk D_t ∈ SD do
    D'_t ← Preprocess(D_t)
    dtype ← None
    if t > 1 then
        dtype, θ ← Detect(D'_t, W_hist, ω, δ)
    end if
    D_train ← D'_t
    if dtype ≠ None then
        ρ ← ρ_dtype
        D_train ← Augment(D'_t, W_hist, M, θ, ρ, dtype)
        δ ← UpdateBoundary(dtype)
    end if
    M ← Train(M, D_train, α)
    W_hist.push(D'_t)
    if |W_hist| > W_h then
        W_hist.pop()
    end if
end for
return M
```

4 Experiment

To investigate the performance of the DACD method in various concept drift scenarios, its performance in practical applications will be comprehensively evaluated based on five dimensions: average real-time accuracy, final cumulative accuracy, convergence speed, robustness, and running time. The experiments in this paper were conducted on Microsoft Windows 11 (Version 24H2), with an Intel Core i7-12700H 2.30 GHz CPU, 40GB of RAM, and an NVIDIA GeForce RTX 3060 Laptop GPU (6GB VRAM). The DACD method was compiled and run using PyCharm 2024.3.1.1, with CUDA 12.6.85 for acceleration.

All experiments follow the temporal order of the data stream. For each incoming chunk, the current model is first evaluated on that chunk to obtain the chunk-level accuracy, and the model is then updated using the current chunk together with eligible historical chunks that have already been observed. Future chunks are not used for either evaluation or model updating at the current step.

4.1 Datasets

The datasets used in the experiments consist of six datasets with gradual, abrupt, and incremental drift characteristics, created using the Massive Online Analysis (MOA) [21] open-source framework for data stream mining, as well as four real-world datasets involving concept drift from common scenarios. Information regarding the datasets used is presented in Table 1.

Among these, the LED_abrupt dataset, created using the MOA framework, is intentionally configured with a single, distinct drift point precisely at the 50% mark of the data stream. This setup creates a controlled benchmark scenario designed specifically to evaluate a model's ability to detect and adapt to a sudden, significant change in the data's underlying distribution.

4.2 Parameter settings

The relevant parameters for the method are set as follows:

1. **Data chunk size** S
   This parameter defines the size of each data chunk into which the dataset is divided during preprocessing. An

Table 1 Experimental data samples

Data Source	Dataset DS	Samples	Features	Classes C	Drift Type	Drift (%)
Simulated	Sea	100000	3	2	Gradual	25, 50, 75
	RBFBlips	100000	20	4	Abrupt	25, 50, 75
	LED_abrupt	100000	24	10	Abrupt	50
	LED_gradual	100000	24	10	Gradual	25, 50, 75
	Tree	100000	30	10	Abrupt	25, 50, 75
	Hyperplane	100000	10	2	Incremental	–
Real-world	Electricity	45300	6	2	–	–
	Kddcup99	494000	41	23	–	–
	Covertype	581000	54	7	–	–
	Weather	95100	9	3	–	–

Note: "-" indicates that the drift type or drift points are unknown

excessively large chunk size may obscure local changes in the data distribution, while a chunk size that is too small can lead to overly frequent model parameter updates, affecting model performance. A suitable chunk size can strike a balance between computational efficiency and the model's adaptability.

In this paper, $S = 500$; a detailed parameter sensitivity analysis is provided in Appendix B.

2. **Drift detection threshold ω and drift boundary δ**
   The drift detection threshold ω is used to determine whether a feature has drifted; when a feature's drift score exceeds ω, the feature is considered to have drifted. The drift boundary δ is used to adjust the sensitivity of drift detection when calculating the cumulative sum statistic. Based on the parameter sensitivity analysis in Appendix B, this paper sets the initial values to $\omega = 4.5$ and $\delta_0 = 0.3$. The subsequent adjustment of δ follows (7) and is based only on the most recently detected drift type: after an abrupt drift, δ is set to 0.1 for the next detection window; after a gradual drift, δ is set to 0.3.
3. **Learning rate α and weight decay coefficient λ**
   In the AdamW optimizer, the learning rate α controls the step size of model parameter updates. If the learning rate is too large, the model may overshoot the optimal solution, leading to non-convergence; if it is too small, the model's convergence speed will be very slow.
   The weight decay coefficient λ helps prevent model overfitting by adding a penalty term to the loss function, which keeps the model's parameters from becoming excessively large. This paper adopts commonly used parameter values for AdamW: $\alpha = 0.001$ and $\lambda = 0.01$.
4. **Batch size B and number of epochs E**
   The batch size indicates the number of samples used in each training iteration. A larger batch size may cause the model to get stuck in a local optimum and can lead to insufficient memory, while a smaller batch size reduces training efficiency and requires more epochs. The number of epochs represents the number of times the entire dataset is trained by the model. Excessively large or small values for these parameters can lead to overfitting or underfitting, respectively.
   Both parameters need to be adjusted appropriately based on the experimental environment. The default parameters used in this paper are $B = 256$ and $E = 50$.
5. **Historical data chunk window size W_h**
   This parameter is used to limit the maximum number of historical data chunks in the adaptive sample processing strategy. This helps to avoid interference in model training from using too much outdated historical data and also reduces the computational burden. In this paper, $W_h = 10$.
6. **Sampling ratio ρ**
   This parameter represents the sampling ratio for historical samples, which is adjusted based on the type of drift in the training sample augmentation phase. In abrupt drift scenarios, historical samples differ significantly from the current data distribution, so a smaller sampling ratio is used to reduce interference. In gradual drift scenarios, historical samples still share stronger similarities with the current distribution, so a larger sampling ratio is used to support smoother adaptation.
   This paper sets $\rho = 0.3$ for abrupt drift and $\rho = 0.6$ for gradual drift.

4.3 Evaluation metrics

1. **Average real-time accuracy ($Avgracc$)**
   Average real-time accuracy represents the mean of the chunk-level accuracy ($racc$) over T sequential evaluation periods. In this paper, real-time accuracy refers to the sequential evaluation of data chunks along the stream, rather than sample-by-sample online updating. It reflects the model's stream-wise predictive performance and its ability to adapt to the phenomenon of concept drift.

Let n_t be the number of samples correctly classified within the current evaluation period t, and $|n|$ be the total number of samples within t. A higher value indicates better chunk-level classification performance for the model as the stream evolves.

$$\text{racc}_t = \frac{n_t}{|n|} \tag{24}$$

$$Avgracc = \frac{1}{T}\sum_{t=1}^{T} \text{racc}_t \tag{25}$$

2. **Final Cumulative Accuracy** ($Fincacc$)
Final cumulative accuracy represents the final value of the cumulative accuracy ($cacc$) from each time period, which is the cumulative accuracy at the end time, $Fincacc = cacc_T$. It reflects the overall classification performance of the model.
Let T_t be the total duration from the start time to the current time period. A higher value indicates better overall classification performance for the model.

$$cacc = \frac{\sum_{t=1}^{T_t} n_t}{|n| \cdot T_t} \tag{26}$$

$$Fincacc = cacc_T = \frac{1}{|n| \cdot T}\sum_{t=1}^{T} n_t \tag{27}$$

3. **Recovery Speed under Accuracy** (RSA)
Recovery Speed under Accuracy represents the product of the number of steps ($Step$) taken for the model to reach stability after a concept drift has occurred (from the drift point to the stable point), and the average error rate ($Avgerr$) over the subsequent k time periods starting from the stable point.

$$RSA = Step \times Avgerr \tag{28}$$

A time period is considered a stable point if the absolute difference between its real-time accuracy and the average accuracy of the subsequent k time periods is less than a stability determination threshold γ, and if the difference between the average real-time accuracies of the two halves of the subsequent k points is less than $\frac{\gamma}{2}$:

$$\begin{cases} |\text{racc}_t - \text{Avgerr}| < \gamma \\ \frac{k}{2}\left|\sum_{i=0}^{\frac{k}{2}-1} \text{racc}_t - \sum_{i=\frac{k}{2}}^{k-1} \text{racc}_{t+i}\right| < \frac{\gamma}{2} \end{cases} \tag{29}$$

Let $racc = \{racc_1, racc_2, \ldots, racc_k\}$. The average error rate $Avgerr$ is then :

$$Avgerr = \frac{\sum_{i=1}^{k}(1 - racc_i)}{k} \tag{30}$$

4. **Robustness** (R)
Robustness is used to evaluate the generalization ability of a model. In this paper, the generalization ability of each method is analyzed by comparing their real-time accuracy. Let the algorithm being evaluated be ALG. Its robustness on dataset DS is:

$$R_{ALG}(DS) = \frac{racc_{ALG}(DS)}{min\ racc(DS)} \tag{31}$$

where min racc(DS) represents the minimum average real-time accuracy among all methods on dataset DS. The overall robustness of a method is the sum of its robustness across all N experimental datasets. A higher value indicates stronger generalization ability for the algorithm.

$$R_{ALG} = \sum_{i=1}^{N} R_{ALG}(DS) \tag{32}$$

4.4 Results and analysis

4.4.1 Model classification accuracy results and analysis

To investigate the performance of the DACD method in different scenarios, various types of methods for handling concept drift in streaming data were selected for comparison. These include methods based on ensemble learning frameworks: TAEL [22], which is based on a two-stage adaptive approach; DWCDS [23], which utilizes a dual-window mechanism; ARF [24], based on adaptive random forests; and SRP [25], based on random subspaces and online bagging. Additionally, methods based on neural network models were included: HBP [26], which is based on the Hessian matrix, and DenseNet [27], based on convolutional neural networks.

Table 2 and Fig. 5 present the comparison of average real-time accuracy and cumulative accuracy of the different methods on the experimental datasets, respectively. The experimental data shows that the DACD method can exhibit excellent classification performance across various types of drift scenarios.

DACD performs particularly well when handling data with non-linear characteristics (e.g., the RBFBlips dataset) and binary features (e.g., the LED datasets). The AdamW optimizer and learning-rate scheduler support stable parameter updates, while the optional embedding layer designed

Table 2 Ranking of average real-time accuracy of different methods on experimental datasets

Dataset	TAEL	DWCDS	ARF	SRP	HBP	DenseNet	DACD
Sea	0.8378 (2)	0.7499 (5)	0.6706 (6)	0.6012 (7)	0.7771 (3)	0.7544 (4)	0.8513 (1)
RBFBlips	0.9549 (2)	0.8214 (7)	0.8327 (5)	0.8294 (6)	0.8350 (4)	0.9452 (3)	0.9897 (1)
LED_abrupt	0.6229 (2)	0.3700 (5)	0.1025 (6)	0.0991 (7)	0.5692 (4)	0.6116 (3)	0.7220 (1)
LED_gradual	0.6199 (3)	0.3804 (5)	0.1024 (6)	0.1007 (7)	0.5650 (4)	0.6208 (2)	0.7426 (1)
Tree	0.6669 (2)	0.5558 (3)	0.4899 (4)	0.3459 (6)	0.1432 (7)	0.3499 (5)	0.6918 (1)
Hyperplane	0.9110 (2)	0.6812 (7)	0.7733 (5)	0.6866 (6)	0.8692 (4)	0.8905 (3)	0.9403 (1)
Electricity	0.7193 (4)	0.7346 (3)	0.7536 (2)	0.6287 (6)	0.5676 (7)	0.6687 (5)	0.8431 (1)
Kddcup99	0.9449 (4)	0.9829 (3)	0.9938 (1)	0.5193 (7)	0.7670 (6)	0.8756 (5)	0.9860 (2)
Covertype	0.7526 (5)	0.8486 (2)	0.8990 (1)	0.8193 (4)	0.6465 (7)	0.6724 (6)	0.8423 (3)
Weather	0.8970 (5)	0.9566 (2)	0.9413 (3)	0.9244 (4)	0.8139 (7)	0.8405 (6)	0.9675 (1)
Overall Ranking	3.1 (2)	4.2 (4)	3.9 (3)	6.0 (7)	5.3 (6)	4.2 (4)	1.3 (1)

Note: Red indicates the highest average real-time accuracy and blue indicates the second highest average real-time accuracy on each dataset

for binary features enhances the model's ability to capture complex feature relationships by adding learnable parameters.

On the real-world Electricity dataset, DACD achieves the highest average real-time accuracy in Table 2, with an accuracy of 0.8431. This is higher than ARF (0.7536), the second-ranked method, by 0.0895. Nevertheless, the Electricity stream contains not only concept drifts but also strong recurring patterns (e.g., daily or weekly cycles in electricity consumption). These recurring patterns remain a limitation for the current window CUSUM detector because they may trigger redundant model adaptations even when the average accuracy is high.

As shown in Table 2, DACD does not achieve the highest average real-time accuracy on Kddcup99 and Covertype. Kddcup99 contains a large number of samples and multiple classes, and its drift type is not explicitly known. In this case, the chunk-level drift detector and historical sample reuse may smooth some fine-grained local distribution changes, while ARF can benefit from its ensemble tree structure in capturing class-specific decision boundaries. Therefore, DACD ranks second on Kddcup99 but still remains close to the best-performing method. For Covertype, the high-dimensional real-world feature space and unknown drift characteristics make drift-type-aware historical sample selection more difficult, which may reduce the alignment between augmented samples and the current data distribution. Nevertheless, DACD still ranks third on this dataset, indicating competitive performance under large-scale real-world stream conditions. These observations suggest that very large real-world streams with unknown drift types remain challenging cases for DACD, especially when local distribution changes are difficult to separate from stable historical patterns.

In summary, DACD demonstrates strong performance in most concept drift scenarios through dynamic parameter adjustment and multi-component integration. However, when faced with complex data streams where recurring patterns coexist with true drifts, the characteristics of its drift detection module still require further improvement.

4.4.2 Ablation study

To analyze the contribution of each component, an ablation study was conducted on four representative datasets covering gradual drift, abrupt drift, binary-feature data, and a real-world stream. The results are shown in Table 3.

The ablation results show that disabling the window CUSUM detector causes the largest average decrease across the four datasets, reducing the mean accuracy from 0.7822 to 0.6192. Removing adaptive sample processing also consistently reduces performance, especially on Electricity and LED_gradual, which indicates that temporally constrained historical augmentation helps maintain stable learning under drift. Removing the class-weighted loss has a particularly large effect on LED_gradual, where the accuracy drops from 0.7426 to 0.4917, showing the importance of class imbalance handling. The optional embedding layer has a smaller effect on Sea and Electricity but remains useful for LED_gradual, where binary features are more prominent.

4.4.3 Method significance difference

In this paper, the Friedman's test [28], a non-parametric statistical method, was selected to test whether the differences among the models in specific performance metrics are statistically significant .

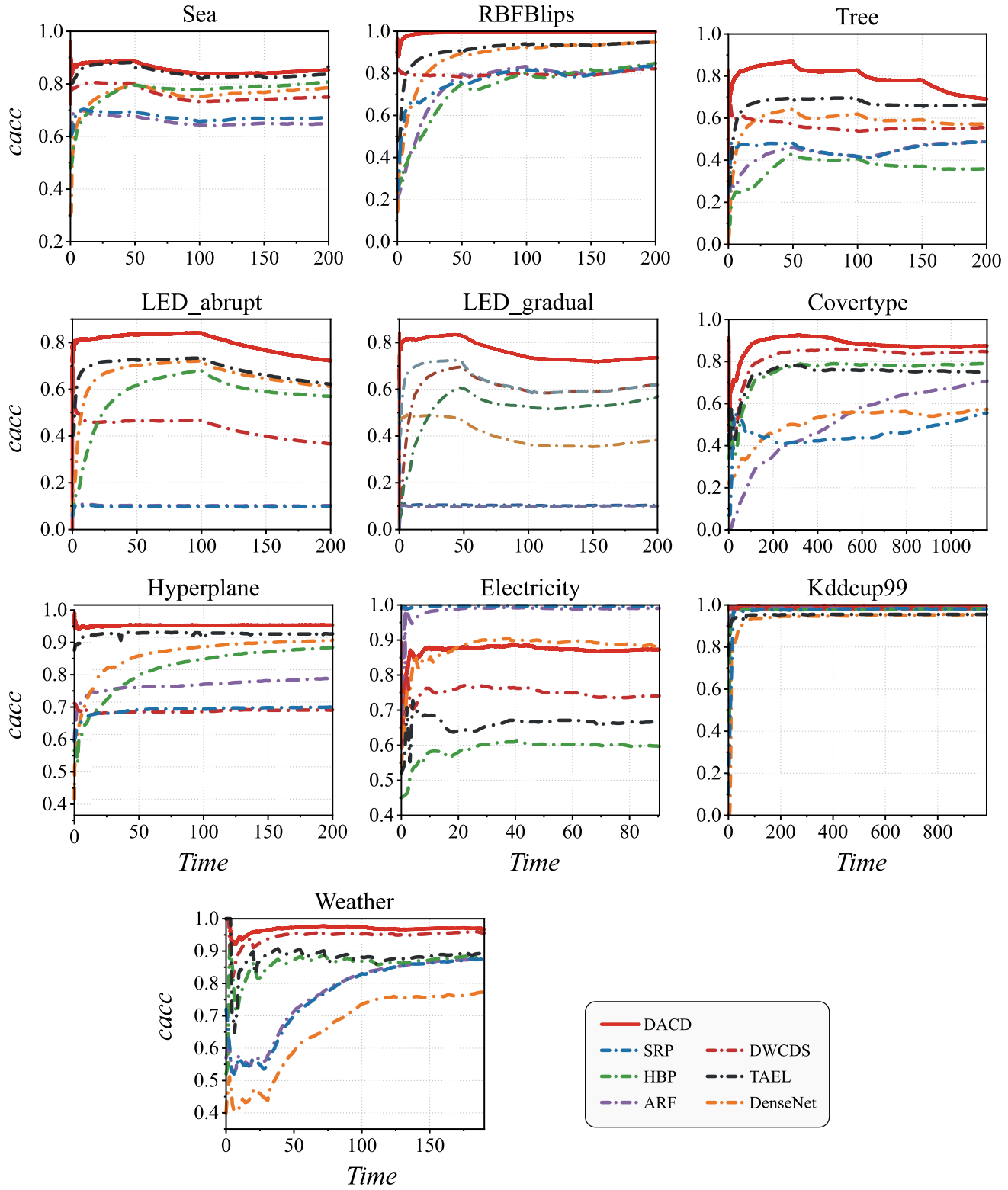


Fig. 5 Comparison of cumulative accuracy for different methods on the experimental datasets

Table 3 Ablation results of DACD components on representative datasets

Dataset	DACD	w/o CUSUM	w/o Embedding	w/o ASP	w/o CW-Loss
Sea	0.8513 (1)	0.7853 (5)	0.8507 (2)	0.8207 (4)	0.8315 (3)
Tree	0.6918 (1)	0.4032 (5)	0.6723 (2)	0.6215 (4)	0.6408 (3)
Electricity	0.8431 (1)	0.7561 (5)	0.8382 (2)	0.7619 (4)	0.7736 (3)
LED_gradual	0.7426 (1)	0.5320 (4)	0.6173 (3)	0.6205 (2)	0.4917 (5)

Note: ASP denotes adaptive sample processing, and CW-Loss denotes class-weighted loss. Red indicates the highest average real-time accuracy and blue indicates the second highest average real-time accuracy on each dataset

This experiment used $K = 7$ methods and $N = 10$ datasets. Let r_i^j be the rank of the j-th method on the i-th dataset. The average rank of the j-th method is:

$$R_j = \frac{1}{N}\sum_{i=1}^{N} r_i^j \tag{33}$$

The null hypothesis H_0 assumes that all methods have the same performance. Under this assumption, when K and N are sufficiently large, the Friedman statistic τ_F follows an F-distribution with $K-1$ and $(K-1)(N-1)$ degrees of freedom:

$$\begin{aligned} \tau_F &= \frac{(N-1)\tau_{\chi^2}}{N(K-1)-\tau_{\chi^2}} \quad \text{where} \\ \tau_{\chi^2} &= \frac{12N}{K(K+1)}\left[\sum_{j=1}^{K} R_j^2 - \frac{K(K+1)^2}{4}\right] \end{aligned} \tag{34}$$

If the calculated statistic exceeds the critical value of the F-distribution at a specific significance level, the null hypothesis H_0 is rejected, indicating that there are significant differences in the rank sums of the methods, which means there are significant differences in their performance. Otherwise, there is no significant difference in the performance of all methods.

In this experiment, at a significance level of $\alpha = 0.05$, the calculated $\tau_{\chi^2} = 29.74$ and the Friedman statistic $\tau_F = 8.85$. This value is greater than the critical F-distribution value $\tau_F^{0.05} = 2.27$. Therefore, the null hypothesis H_0 is rejected, indicating that the performance of the methods is significantly different.

The Bonferroni-Dunn test [29] is a statistical method for multiple comparisons that can visually reflect the differences between methods. If the difference between the average ranks of two methods is greater than the critical difference (CD), the methods are considered to have a statistically significant difference. If the difference is smaller than CD, the methods may still differ in their average ranks, but the difference does not reach the statistical significance threshold under this test:

$$CD = q_\alpha\sqrt{\frac{K(K+1)}{6N}} \tag{35}$$

At a significance level of $\alpha = 0.05$, the calculated CD for this experimental setup is 2.5486. As shown in Fig. 6, the average ranks used in the Bonferroni-Dunn comparison are DACD = 1.3, TAEL = 3.1, ARF = 3.9, DWCDS = 4.2, DenseNet = 4.2, HBP = 5.3, and SRP = 6.0. The rank differences between DACD and ARF, DWCDS, DenseNet, HBP, and SRP are greater than the critical difference; therefore, DACD is significantly better than these methods under this test. The rank difference between DACD and TAEL is 1.8, which is smaller than $CD = 2.5486$. This result still indicates a difference in the average ranks of the two methods, but the difference is not large enough to be considered statistically significant under the Bonferroni-Dunn test. Nevertheless, this aggregate test result does not imply that the two methods behave equivalently on every dataset; as reported in Table 2, DACD still achieves stronger performance than TAEL on several individual data streams, particularly on datasets with abrupt or binary-feature drift patterns.

4.4.4 Model convergence analysis

To compare how quickly models can adapt to new data distributions after a concept drift occurs, this experiment primarily analyzes the RSA of each method at the known drift points

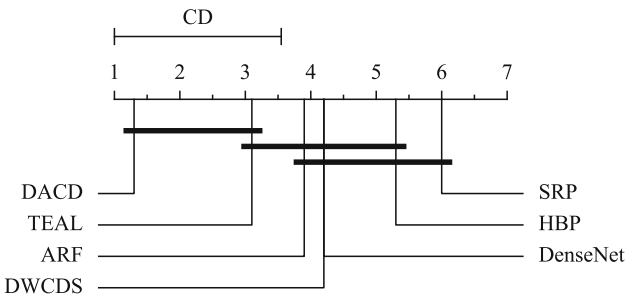


Fig. 6 Bonferroni-Dunn comparison based on the average ranks of the seven methods

in the datasets (Sea, RBFBlips, LED_abrupt, LED_gradual, Tree). As indicated in Table 1, the LED_abrupt dataset is configured with a single drift point; therefore, the analysis for this dataset was conducted exclusively at the 50% mark. Each drift point corresponds to a convergence point.

As shown in Table 4, DACD demonstrates competitive and consistent convergence performance compared with the other methods. Its performance is stable across datasets with distinct drift characteristics, including gradual drift (Sea) and multiple abrupt drifts (RBFBlips, LED_abrupt, Tree). This indicates an ability to adapt and stabilize its accuracy after different types of concept changes. In contrast, several comparison methods fail to converge or converge more slowly, especially in the face of abrupt changes. This difference highlights the effectiveness of DACD's integrated optimization mechanisms for dynamic streaming data environments.

For LED_gradual, DACD is not the fastest at the three drift points because gradual drift evolves across multiple chunks. The use of temporally reliable historical samples helps maintain stable long-term adaptation, but may slightly delay immediate recovery after each drift point. For Tree, abrupt local changes can favor methods that adjust more aggressively to the latest distribution, such as DenseNet. In contrast, DACD balances local recovery with stable retraining through drift detection and historical sample augmentation. Thus, these local non-best results reflect a trade-off between recovery speed and stability, while DACD still achieves the best overall convergence rank in Table 4.

4.4.5 Robustness analysis of the experimental method

Robustness as defined by (32) is a crucial metric for evaluating a method's stability across different types of scenarios. As shown in Fig. 7, DACD exhibits high overall robustness, with a score significantly higher than the comparison methods.

Simulated datasets feature diverse changes in data distribution. For example, the RBFBlips dataset has non-linear characteristics; DACD can accurately identify changes and capture features, maintaining stable performance and thus improving its robustness on this dataset. Real-world datasets present challenges such as noise, recurring patterns, and varied feature dimensions. Taking the Electricity dataset as an example, DACD combines drift detection, temporally constrained historical augmentation, and class-weighted training to maintain robust performance despite recurring demand patterns.

In conclusion, DACD's integrated use of multiple techniques results in strong adaptability and stability across different types of datasets, supporting its application to streaming data concept drift scenarios.

4.4.6 Running time analysis

To further evaluate computational efficiency, Table 5 reports the running time of DACD and the comparison methods on the experimental datasets. All timing experiments

Table 4 Results of the recovery speed under accuracy at drift points for different methods

Dataset	Drift Point	TAEL	DWCDS	ARF	SRP	HBP	DenseNet	DACD
Sea	25%	0.45 (1)	0.67 (3)	4.54 (6)	- (7)	0.77 (4)	2.12 (5)	0.51 (2)
	50%	1.97 (4)	0.25 (1)	16.52 (6)	- (7)	0.51 (3)	2.33 (5)	0.26 (2)
	75%	1.50 (4)	0.45 (3)	0.32 (2)	- (6)	1.82 (5)	- (6)	0.31 (1)
RBFBlips	25%	0.03 (1)	0.56 (4)	- (6)	- (6)	1.14 (5)	0.21 (3)	0.04 (2)
	50%	0.29 (3)	0.85 (5)	- (6)	- (6)	0.83 (4)	0.19 (2)	0.14 (1)
	75%	0.01 (1)	0.16 (4)	- (6)	- (6)	0.21 (5)	0.05 (3)	0.02 (2)
LED_abrupt	50%	5.28 (3)	3.70 (1)	- (6)	- (6)	20.20 (5)	9.07 (4)	3.97 (2)
LED_gradual	25%	8.25 (4)	2.20 (1)	- (6)	- (6)	13.12 (5)	5.39 (2)	6.72 (3)
	50%	5.32 (2)	1.95 (1)	- (6)	- (6)	10.86 (5)	8.62 (4)	5.82 (3)
	75%	3.71 (2)	- (5)	- (5)	- (5)	7.42 (4)	2.22 (1)	4.73 (3)
Tree	25%	3.85 (5)	9.29 (6)	3.71 (4)	- (7)	3.56 (3)	0.21 (1)	3.52 (2)
	50%	3.92 (4)	4.69 (5)	4.84 (6)	8.52 (7)	0.87 (2)	0.19 (1)	3.58 (3)
	75%	3.85 (4)	4.70 (5)	- (7)	5.26 (6)	1.75 (2)	0.05 (1)	3.76 (3)
Average Ranking		2.92 (2)	3.38 (3)	5.54 (5)	6.23 (6)	4.00 (4)	2.92 (2)	2.23 (1)

Note: Red indicates the best recovery speed and blue indicates the second best recovery speed at each drift point. "-" indicates the method did not converge at this point

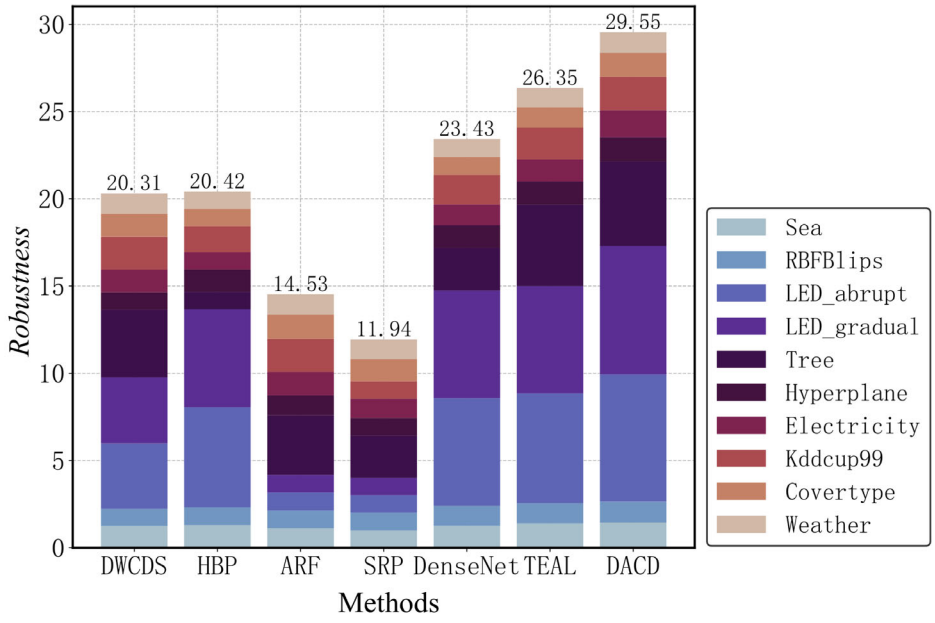


Fig. 7 Comparison of the overall robustness of different methods

were carried out under the same hardware and software environment described above, and the same data-loading, preprocessing, and timing procedure was used for all methods. The comparison methods were reproduced according to the algorithm descriptions and experimental settings reported in their corresponding papers, so the runtime results were obtained under a consistent evaluation protocol. In the table, smaller values indicate faster execution, and the values in parentheses denote the rank of each method on the corresponding dataset.

As shown in Table 5, DACD achieves the best average running-time rank among the seven compared methods. It obtains the shortest running time on Sea, RBFBlips, LED_abrupt, LED_gradual, Hyperplane, and Electricity, and ranks second on Tree. TAEL is the fastest method on Tree, while DWCDS is the fastest method on Kddcup99, Covertype, and Weather. The results indicate that DACD maintains competitive computational efficiency while preserving the accuracy and robustness advantages shown in the preceding experiments. On larger real-world datasets such as Covertype and Weather, DACD is slower than DWCDS and HBP because neural network retraining and adaptive sample processing introduce additional computation, but its overall running-time ranking remains the best among the compared methods.

4.4.7 Limitation analysis of the method

The DACD method still has certain limitations in handling concept drift. The current drift detection mechanism

Table 5 Running time comparison of different methods (Unit: S)

Dataset	TAEL	DWCDS	ARF	SRP	HBP	DenseNet	DACD
Sea	29.076 (2)	29.191 (3)	1338.530 (7)	293.210 (6)	57.470 (4)	82.660 (5)	28.730 (1)
RBFBlips	7.080 (2)	36.000 (3)	346.660 (6)	525.080 (7)	58.680 (4)	78.570 (5)	6.921 (1)
LED_abrupt	3.702 (2)	38.567 (4)	912.360 (6)	1238.550 (7)	9.910 (3)	86.180 (5)	3.252 (1)
LED_gradual	7.406 (2)	37.464 (4)	900.630 (6)	1264.890 (7)	10.070 (3)	78.240 (5)	7.280 (1)
Tree	13.410 (1)	46.545 (3)	665.460 (6)	896.040 (7)	58.140 (4)	85.020 (5)	13.650 (2)
Hyperplane	30.000 (2)	31.783 (3)	385.300 (6)	648.170 (7)	68.660 (4)	83.630 (5)	29.460 (1)
Electricity	13.268 (2)	22.629 (3)	118.750 (6)	216.710 (7)	31.370 (4)	43.130 (5)	11.137 (1)
Kddcup99	198.055 (2)	155.564 (1)	1165.840 (6)	1697.720 (7)	255.390 (4)	421.460 (5)	199.122 (3)
Covertype	403.567 (3)	170.406 (1)	1477.610 (6)	2049.100 (7)	292.070 (2)	533.490 (5)	417.253 (4)
Weather	111.793 (4)	36.697 (1)	159.260 (6)	220.970 (7)	53.600 (2)	79.190 (3)	121.654 (5)
Average Ranking	2.2 (2)	2.6 (3)	6.1 (6)	6.9 (7)	3.4 (4)	4.8 (5)	2.0 (1)

is primarily designed for abrupt and gradual drifts and experiences a response delay when dealing with incremental drift. This is because the gradual, cumulative changes in its feature distribution are easily overlooked by the window CUSUM algorithm, which may cause model updates to lag behind the actual drift process. For cyclical drifts, although the algorithm can detect conceptual changes, it struggles to identify the periodic patterns of the drift, potentially leading to redundant updates for recurring drift patterns and affecting model stability. This limitation is especially relevant to datasets such as Electricity: although DACD obtains the best average real-time accuracy on this dataset in Table 2, recurring demand patterns can still cause unnecessary drift alarms.

Furthermore, when faced with incremental drift in high-dimensional feature spaces, the adaptive sample processing strategy's class-proportional sampling may not accurately capture the changing correlations between features. This can lead to a reduced match between the augmented samples and the current drift pattern. At the same time, the algorithm's adaptive parameter adjustments for different drift types (such as the boundary parameter for abrupt drift) rely on pre-set values. In complex mixed-drift scenarios, errors in drift type classification could lead to improper parameter configurations, thereby affecting detection accuracy and model performance. A concrete improvement is to add a periodicity-aware drift discriminator before triggering retraining, for example by combining window CUSUM with autocorrelation or seasonal decomposition tests, and to use a separate incremental-drift buffer for samples whose changes accumulate slowly.

5 Conclusion

The DACD method proposed in this paper addresses the challenges of concept drift in streaming data by systematically integrating window CUSUM drift detection, adaptive sample processing, and a neural network model. Comprehensive experimental validation on multiple simulated and real-world datasets demonstrates that the proposed method achieves strong performance in terms of average real-time accuracy, final cumulative accuracy, convergence speed, robustness, and running time. DACD shows particular proficiency in handling non-linear data distributions (e.g., the RBFBlips dataset) and data with binary features (e.g., the LED datasets), a strength attributable to the integration of drift detection, adaptive sample processing, class-weighted training, and the optional embedding layer.

However, the analysis also reveals limitations, particularly with data streams characterized by strong recurring patterns, as seen in the Electricity dataset, where the CUSUM detector's sensitivity can lead to redundant updates due to misidentification of cyclical fluctuations as concept drifts. Furthermore, the method exhibits a delayed response to incremental drifts and lacks a mechanism to specifically model cyclical drift patterns.

Future research will therefore focus on enhancing the drift detection mechanism to distinguish between true concept drifts and recurring or cyclical patterns, for example by adding periodicity tests before retraining and by separating recurring-pattern alarms from one-time distribution shifts. We also aim to develop specialized strategies for incremental and mixed-drift scenarios, including dynamic windows and incremental-drift sample buffers. Additionally, future work will include more detailed memory comparisons and hardware-normalized efficiency analysis against baseline methods.

Appendix A: Complexity analysis of the DACD method

The model complexity of the DACD method is primarily distributed across four core stages: data preprocessing, window CUSUM detection, model training and prediction, and adaptive sample processing. The following is a detailed complexity analysis.

1. **Data Preprocessing**
   Data preprocessing, as the initial stage of the algorithm, needs to complete standardization and label remapping. Specifically, for a dataset containing N samples and Y-dimensional features, the standardization operation requires iterating through all features of each sample to calculate the mean and standard deviation, a process with a time complexity of $O(NY)$. Label remapping requires mapping the original labels to a continuous sequence starting from 0, with a time complexity of $O(N)$. Therefore, the overall time complexity is dominated by standardization, which is $O(NY)$.
   In terms of space occupation, the preprocessed data needs to be stored completely, occupying $O(NY)$ space. Additionally, the mean and standard deviation parameters required for standardization occupy $O(Y)$ space, and the label mapping table occupies $O(C)$ space (where C is the number of classes). Thus, the total space complexity is $O(NY + Y + C)$.
2. **Window CUSUM Detection**
   Concept drift detection employs the window CUSUM algorithm. First, statistics are compiled for the W historical data chunks within the window to calculate the mean and standard deviation of the reference distribution. This step requires iterating through the n samples and their Y-dimensional features in each of the W chunks, resulting in a time complexity of $O(nWY)$. Next, the standardized

differences for the n samples in the current data chunk are calculated, and the CUSUM statistics are accumulated to generate drift scores, a process with a time complexity of $O(nY)$. Finally, a multi-feature joint judgment is used to determine if a drift has occurred, with a time complexity of $O(Y)$. Therefore, the overall time complexity is $O((nY)(W+1))$.

In terms of space, it is necessary to store the statistics (mean, standard deviation) of the historical chunks within the window, as well as the CUSUM statistics and drift scores of the current chunk, leading to a total space complexity of $O(Y(W+n))$.

3. **Model Training and Prediction**
   The model training utilizes a neural network architecture. The training process includes forward propagation, loss calculation, backpropagation, and parameter updates. For a model with an input dimension of Y, a hidden layer dimension of H, and an output class number of C, the time complexity of a single forward propagation is $O(H(Y+C))$, and the complexity of backpropagation is of the same order. If training is conducted for E epochs, processing N samples per epoch with a batch size of B, the total number of training iterations is $\frac{N}{B}$. Thus, the time complexity of the training phase is $O(\frac{N}{B}(Y+C)EH)$. During the prediction phase for M test samples, the time complexity is $O((Y+C)HM)$.
   In terms of space complexity, the model parameters (weights and biases) occupy $O((Y+C)H)$ space. During training, it is necessary to store intermediate activation values (such as the hidden layer output), which occupies $O(\frac{N}{B}H)$ space. Including the storage overhead for the training data and labels, $O(N(Y+1))$, the total space complexity is $O(N(Y+1)+(Y+C+\frac{N}{B})H)$.
4. **Adaptive Sample Processing**
   Adaptive sample processing includes historical chunk filtering, sampling, and sample augmentation. First, historical chunks are filtered (retaining a maximum of M chunks), with each chunk containing n samples, resulting in a time complexity of $O(nMY)$. Next, the filtered historical chunks are augmented through class-proportional sampling. This requires iterating through the n_H samples in the historical data to determine the class distribution, with a time complexity of $O(n_H)$. Then, based on the N samples in the current chunk, class priorities are calculated and the augmentation is completed, with a time complexity of $O(N+E)$ (where E is the number of augmented samples). Therefore, the overall time complexity is $O(nMY+n_H+N+E)$.
   In terms of space, the augmented samples need to be stored, occupying $O((N+E)Y)$ space. The storage for the filtered historical chunks occupies $O(nMY)$. Thus, the total space complexity is $O((nM+N+E)Y)$.
5. **Comprehensive Analysis of Computational and Storage Costs**
   The neural network operations during the model training phase (especially when the hidden layer dimension H is large) constitute the main time-consuming part, with complexity growing quadratically with the number of features Y and the hidden layer dimension H. The computational overhead of the window CUSUM detection also increases significantly in scenarios with high-dimensional features (large Y) or large windows (large W).

The storage cost bottleneck of DACD lies in the significant space occupied by sample storage during data preprocessing and adaptive sample processing. Especially when the number of historical chunks M and the number of augmented samples are large, the space complexity grows linearly with the sample size. In practical applications, storage pressure can be alleviated through dynamic sample sampling (e.g., limiting the number of historical chunks, reducing the augmentation ratio) and GPU memory optimization (e.g., batch-based computation).

Appendix B: Parameter sensitivity analysis

In the DACD method, parameter settings are crucial to model performance. The data chunk size S affects detection sensitivity and computational efficiency; a chunk that is too small is susceptible to noise interference, while one that is too large delays the response. The drift detection threshold ω and the drift boundary δ determine the model's ability to identify conceptual changes; values that are too small or too large can lead to misjudgments or delayed responses. The following is a specific experimental analysis of parameter sensitivity.

1. **Data Chunk Size** S
   The data chunk size S has a significant impact on the sensitivity of drift detection and computational efficiency. The chunk size determines the granularity at which the model processes the data stream. A chunk size that is too small will lead to overly frequent drift detections, increasing computational overhead and potentially introducing noise interference. Conversely, a chunk size that is too large will reduce detection sensitivity, causing a delayed response to drifts.
   An analysis of the model's average real-time accuracy under different data chunk sizes S, as shown in Table 6, indicates that a chunk size of 500 achieves the best overall average ranking. This is because it achieves a good balance between drift detection sensitivity and model update efficiency. Smaller data chunks can capture abrupt fea-

Table 6 Average real-time accuracy performance of the model under different S

Dataset	Type	Average real-time accuracy			
		S=100	S=500	S=1000	S=2000
Sea	Gradual	0.8447 (2)	0.8513 (1)	0.8098 (4)	0.8210 (3)
RBFBlips	Abrupt	0.9913 (1)	0.9897 (2)	0.9641 (3)	0.9220 (4)
LED_abrupt	Abrupt	0.7060 (2)	0.7220 (1)	0.6603 (3)	0.6432 (4)
LED_gradual	Gradual	0.7335 (3)	0.7426 (1)	0.7394 (2)	0.7287 (4)
Tree	Abrupt	0.6952 (1)	0.6918 (2)	0.6911 (3)	0.6952 (1)
Hyperplane	Incremental	0.9276 (3)	0.9403 (1)	0.9271 (4)	0.9351 (2)
Electricity	–	0.8476 (1)	0.8431 (2)	0.8426 (3)	0.8395 (4)
Kddcup99	–	0.9631 (4)	0.9860 (3)	0.9901 (1)	0.9896 (2)
Covertype	–	0.8419 (4)	0.8423 (3)	0.8435 (2)	0.8483 (1)
Weather	–	0.9662 (3)	0.9675 (2)	0.9678 (1)	0.9583 (4)
Average Ranking		2.4 (2)	1.8 (1)	2.6 (3)	2.9 (4)

Note: "-" indicates the drift type is unknown. Red indicates the best result and blue indicates the second-best result according to the listed ranking

tures in the data stream more promptly, preventing local concept changes from being obscured by an overly large chunk, but they also introduce more noise. Larger data chunks, while smoothing the feature distribution by accumulating more samples, may delay the response to a drift. A size of $S = 500$ ensures the ability to perceive changes in various drift scenarios while reducing statistical fluctuations through a reasonable sample size, allowing the model to maintain stable detection performance across different datasets.

From the perspective of the dynamic adaptation mechanism, a block size strategy based on $S = 500$ can be flexibly adjusted according to the type of drift, further enhancing its advantages. When facing abrupt drift, the block size can be appropriately reduced to increase detection sensitivity, ensuring a response is triggered at the initial stage of the concept change. When dealing with gradual drift, the block size is moderately increased to accumulate sufficient changes in the feature distribution, avoiding misjudgments caused by an insufficient amount of data. In summary, a data chunk size of $S = 500$ was selected for this experiment.

2. **Drift Detection Threshold ω and Drift Boundary δ**
The method uses window CUSUM for concept drift detection, and the performance of this method on different types of drift data varies with different parameter settings. For instance, when detecting abrupt and gradual drifts, different parameter values will affect the method's ability to accurately identify drift points, which in turn affects the subsequent training and prediction accuracy of the model. Table 7 reports the average real-time accuracy under nine (ω, δ) combinations. The values in parentheses indicate the ranking of each parameter combination on the corresponding dataset, where a smaller rank denotes better performance. For readability, the table is split into three blocks by δ, and each block compares the three tested values of ω.

From a mathematical definition standpoint, the model determines whether a concept drift has occurred by calculating a drift score s_j for each feature. The drift score s_j is based on the CUSUM statistics, $CU_{i,j}^{+}$ and $CU_{i,j}^{-}$, which are the positive and negative cumulative sum statistics, respectively.

When $s_j > \omega$, the feature is identified as a drifted feature. When the proportion of drifted features to the total number of features is greater than or equal to 1/3, an overall concept drift is determined to have occurred. Under this mechanism, the threshold ω is inversely proportional to the model's sensitivity: if the value is too small, a lower drift score can satisfy $s_j > \omega$, leading to the misinterpretation of normal data fluctuations as concept drift, which increases the frequency of model adjustments and reduces classification accuracy. If the value is too large, the model cannot adapt to changes in the data distribution in a timely manner, resulting in a decrease in accuracy.

In the experiments, the best overall average rank is obtained at $\omega = 4.5$ and $\delta = 0.3$, so this pair is used as the initial detector setting. The value $\omega = 4.5$ balances sensitivity and stability: it avoids misjudgments caused by over-sensitivity while preventing missed detections due to insufficient sensitivity, thus validating the regulatory effect of the threshold on model sensitivity.

Furthermore, under the same drift detection threshold, $\delta = 0.3$ demonstrates relatively good comprehensive performance across various datasets; therefore, this method defaults to $\delta = 0.3$. Additionally, when $\delta = 0.1$, the average real-time accuracy for abrupt drifts is superior to that of other values. Consequently, the subsequent

Table 7 Average real-time accuracy performance of the model under different ω and δ

Dataset	Type	$\delta = 0.1$			$\delta = 0.2$			$\delta = 0.3$		
		$\omega = 3.0$	$\omega = 4.5$	$\omega = 5.0$	$\omega = 3.0$	$\omega = 4.5$	$\omega = 5.0$	$\omega = 3.0$	$\omega = 4.5$	$\omega = 5.0$
Sea	Gradual	0.7727 (8)	0.8047 (6)	0.8058 (5)	0.7786 (7)	0.8613 (1)	0.7565 (9)	0.8260 (4)	0.8530 (2)	0.8363 (3)
RBFBlips	Abrupt	0.9922 (5)	0.9977 (1)	0.9943 (3)	0.9924 (4)	0.9976 (2)	0.9874 (7)	0.982 (9)	0.9888 (6)	0.9848 (8)
LED_abrupt	Abrupt	0.7107 (4)	0.7220 (1)	0.7172 (2)	0.7047 (6)	0.6896 (9)	0.6995 (8)	0.7077 (5)	0.7144 (3)	0.700 (7)
LED_gradual	Gradual	0.7199 (5)	0.7181 (6)	0.7164 (8)	0.7426 (1)	0.7140 (9)	0.7236 (3)	0.7207 (4)	0.7353 (2)	0.7165 (7)
Tree	Abrupt	0.6918 (1)	0.6914 (2)	0.6766 (4)	0.6200 (8)	0.6182 (9)	0.6256 (7)	0.6698 (5)	0.6682 (6)	0.6792 (3)
Hyperplane	Incremental	0.932 (5)	0.9312 (6)	0.9229 (7)	0.9403 (1)	0.9346 (4)	0.9137 (8)	0.9385 (2)	0.9381 (3)	0.9115 (9)
Electricity	–	0.8579 (8)	0.8507 (9)	0.8836 (1)	0.8583 (7)	0.8757 (3)	0.8802 (2)	0.8644 (6)	0.8720 (4)	0.8654 (5)
Kddcup99	–	0.9954 (4)	0.9956 (3)	0.9947 (5)	0.9873 (9)	0.994 (7)	0.9945 (6)	0.9929 (8)	0.9960 (1)	0.9958 (2)
Covertype	–	0.8129 (7)	0.8741 (3)	0.8224 (6)	0.8121 (8)	0.8416 (5)	0.7957 (9)	0.8526 (4)	0.8751 (2)	0.8823 (1)
Weather	–	0.9185 (8)	0.9429 (6)	0.9616 (3)	0.9349 (7)	0.9606 (4)	0.9644 (2)	0.9179 (9)	0.9675 (1)	0.9456 (5)
Gradual Drift Average Ranking		6.5 (5)	6.0 (4)	6.5 (5)	4.0 (2)	5.0 (3)	6.0 (4)	4.0 (2)	2.0 (1)	5.0 (3)
Abrupt Drift Average Ranking		3.3 (3)	1.3 (1)	3.0 (2)	6.0 (5)	6.7 (7)	7.3 (8)	6.3 (6)	5.0 (4)	6.0 (5)
Overall Average Ranking		5.5 (6)	4.3 (2)	4.4 (3)	5.8 (8)	5.3 (5)	6.1 (9)	5.6 (7)	3.0 (1)	5.0 (4)

Note: "-" indicates the drift type is unknown. Red indicates the best result and blue indicates the second-best result according to the listed ranking

Table 8 Sensitivity analysis of the drift severity threshold θ

Dataset	Type	Average real-time accuracy				
		$\theta = 2.0$	$\theta = 2.5$	$\theta = 3.0$	$\theta = 3.5$	$\theta = 4.0$
Sea	Gradual	0.8493 (5)	0.8495 (4)	0.8513 (1)	0.8510 (3)	0.8512 (2)
RBFBlips	Abrupt	0.9873 (4)	0.9877 (2)	0.9897 (1)	0.9876 (3)	0.9866 (5)
LED_abrupt	Abrupt	0.7182 (5)	0.7193 (2)	0.7220 (1)	0.7191 (3)	0.7190 (4)
LED_gradual	Gradual	0.7359 (5)	0.7380 (3)	0.7426 (1)	0.7386 (2)	0.7378 (4)
Tree	Abrupt	0.6903 (5)	0.6907 (4)	0.6918 (1)	0.6915 (2)	0.6910 (3)
Hyperplane	Incremental	0.9339 (5)	0.9348 (4)	0.9403 (1)	0.9384 (2)	0.9383 (3)
Electricity	–	0.8413 (5)	0.8446 (1)	0.8431 (2)	0.8424 (3)	0.8414 (4)
Kddcup99	–	0.9821 (5)	0.9857 (2)	0.9860 (1)	0.9852 (3)	0.9843 (4)
Covertype	–	0.8212 (5)	0.8354 (3)	0.8423 (2)	0.8437 (1)	0.8276 (4)
Weather	–	0.9672 (2)	0.9650 (5)	0.9675 (1)	0.9668 (4)	0.9670 (3)
Gradual Drift Average Ranking		5.0 (5)	3.5 (4)	1.0 (1)	2.5 (2)	3.0 (3)
Abrupt Drift Average Ranking		4.7 (4)	2.7 (2)	1.0 (1)	2.7 (2)	4.0 (3)
Overall Average Ranking		4.6 (5)	3.0 (3)	1.2 (1)	2.6 (2)	3.6 (4)

Note: "-" indicates that the drift type is unknown. The values in parentheses denote the ranking of each θ threshold setting on the corresponding dataset. Red indicates the best result and blue indicates the second-best result according to the listed ranking

boundary update follows (7): when the most recent detected drift is abrupt, the next detection window uses $\delta = 0.1$; when the most recent detected drift is gradual, the boundary returns to $\delta = 0.3$. This adjustment is based on the latest detected drift event rather than all historical drift events.

3. **Drift Severity Threshold** θ

The drift severity threshold θ controls the boundary between abrupt and gradual drift after a chunk-level drift event has been confirmed. In the proposed detection rule, an event is classified as abrupt only when $\theta \geq 3.0$ and $L < 0.1S$. The first condition evaluates the intensity of the feature-level distribution change, whereas the second condition ensures that the change is concentrated within a short segment of the current chunk. Therefore, the threshold value of θ directly affects the balance between abrupt-drift sensitivity and gradual-drift stability.

To examine this effect, a sensitivity analysis was conducted by varying the threshold value of θ while keeping the other parameters fixed. In this setting, $S = 500$, $\omega = 4.5$, and $\delta_0 = 0.3$ are used, and the tested values are $\theta \in \{2.0, 2.5, 3.0, 3.5, 4.0\}$. The corresponding results are summarized in Table 8. A smaller θ threshold makes the method more sensitive to abrupt changes, but it may also classify moderate gradual changes as abrupt drift. In contrast, a larger θ threshold makes the method more conservative, which may improve stability on gradual-drift datasets but delay the response to true abrupt drift.

The sensitivity results justify the default threshold setting $\theta = 3.0$. If the θ threshold is too small, gradual drift may be over-classified as abrupt drift, which reduces the use of historical samples and weakens stability during slow distribution changes. If the θ threshold is too large, true abrupt drift may be treated as gradual drift, which delays adaptation and slows accuracy recovery after sudden changes. Therefore, $\theta = 3.0$ is selected because it provides the best overall ranking while maintaining competitive performance on both abrupt- and gradual-drift datasets.

Author Contributions The idea proposal and method design were performed by Qi Hui. Code implementation and data testing were performed by Liu Chang. The first draft of the manuscript was written by Qi Hui and Liu Chang. Writing guidance, revision, and approval were provided by Qi Xiaobo, Shi Ying, and Jiang Gaoxia. All authors read and approved the final manuscript.

Funding This work was supported by the National Natural Science Foundation of China (62276161); the Humanities and Social Sciences Research Foundation of the Ministry of Education (25YJCZH200); the Research on Shanxi Province Basic Research Plan Free Exploration Category (202403021221193); the Cultivation Project for Transformation of Scientific and Technological Achievements of Higher Education Institutions in Shanxi Province (2025Z011); and the Taiyuan Normal University Achievement Transformation and Technology Transfer Base (2023P003).

Data Availability The simulated dataset was generated using MOA. The real-world dataset and core code implementing the DACD method are available upon reasonable request. Researchers interested in accessing these resources should contact the corresponding authors via email, clearly stating their intended research purposes and usage scenarios, to obtain the relevant materials.

Declarations

Conflict of interest We declare that we have no Conflict of interest.

Ethics approval and consent to participate This study did not involve any human or animal subjects, hence ethical approval and consent to participate are not applicable.

Consent for publication We declare that we have obtained the consent of all parties involved in the study.

References

1. Din SU, Yang Q, Shao J, Mawuli CB, Ullah A, Ali W (2024) Synchronization-based semi-supervised data streams classification with label evolution and extreme verification delay. Inf Sci 678:120933. https://doi.org/10.1016/j.ins.2024.120933
2. Zubaroğlu A, Atalay V (2021) Data stream clustering: a review. Artif Intell Rev 54(2):1201–1236. https://doi.org/10.1007/s10462-020-09874-x
3. Din SU, Shao J, Kumar J, Mawuli CB, Mahmud SMH, Zhang W, Yang Q (2021) Data stream classification with novel class detection: a review, comparison and challenges. Knowl Inf Syst 63(9):2231–2276. https://doi.org/10.1007/s10115-021-01582-4
4. Arora S, Rani R, Saxena N (2024) A systematic review on detection and adaptation of concept drift in streaming data using machine learning techniques. WIREs Data Min Knowl Discovery 14(4):1536. https://doi.org/10.1002/widm.1536
5. Darem AA, Ghaleb FA, Al-Hashmi AA, Abawajy JH, Alanazi SM, Al-Rezami AY (2021) An adaptive behavioral-based incremental batch learning malware variants detection model using concept drift detection and sequential deep learning. IEEE Access 9:97180–97196. https://doi.org/10.1109/ACCESS.2021.3093366
6. Chen S, He H (2009) SERA: Selectively recursive approach towards nonstationary imbalanced stream data mining. In: 2009 International joint conference on neural networks, pp 522–529. https://doi.org/10.1109/IJCNN.2009.5178874
7. Street WN, Kim Y (2001) A streaming ensemble algorithm (SEA) for large-scale classification. In: Proceedings of the Seventh ACM SIGKDD international conference on knowledge discovery and data mining, pp 377–382. https://doi.org/10.1145/502512.502568
8. Yan Z, Du H, Ke G, Zhang L, Chen Y-C (2022) Dynamic weighted selective ensemble learning algorithm for imbalanced data streams. J Supercomput 78(4):5394–5419. https://doi.org/10.1007/s11227-021-04084-w
9. Lu Y, Cheung Y-M, Tang Y (2019) Adaptive chunk-based dynamic weighted majority for imbalanced data streams with concept drift. IEEE Trans Neural Netw Learn Syst 31(8):2764–2778. https://doi.org/10.1109/TNNLS.2019.2951814
10. Brzezinski D, Stefanowski J (2013) Reacting to different types of concept drift: the accuracy updated ensemble algorithm. IEEE Trans Neural Netw Learn Syst 25(1):81–94. https://doi.org/10.1109/TNNLS.2013.2251352
11. Liang B, Li G, Dai C (2022) G-mean weighted classification method for imbalanced data stream with concept drift. J Comput Res Dev 59(12):2844–2857. https://doi.org/10.7544/issn1000-1239.20210471
12. Guo H, Cong L, Gao S, Wang W (2023) Adaptive classification method for concept drift based on online ensemble. J Comput Res Dev 60(7):1592–1602. https://doi.org/10.7544/issn1000-1239.202220245
13. Guo H, Zhang Y, Wang W (2024) Dynamical targeted ensemble learning for streaming data with concept drift. IEEE Trans Knowl Data Eng. https://doi.org/10.1109/tkde.2024.3460404
14. Lima M, Neto M, Filho TS, de A. Fagundes RA (2022) Learning under concept drift for regression—a systematic literature review. IEEE Access 10:45410–45429. https://doi.org/10.1109/ACCESS.2022.3169785
15. Kumar S, Kaur P, Gosain A (2022) A comprehensive survey on ensemble methods. In: 2022 IEEE 7th international conference for convergence in technology (I2CT), pp 1–7. https://doi.org/10.1109/I2CT54291.2022.9825269
16. Liu N, Zhao J (2023) Streaming data classification based on hierarchical concept drift and online ensemble. IEEE Access 11:126040–126051. https://doi.org/10.1109/ACCESS.2023.3327637
17. Garcia CM, Abilio R, Koerich AL, Britto ADS, Barddal JP (2025) Concept drift adaptation in text stream mining settings: a systematic review. ACM Trans Intell Syst Technol 16(2):27–12767. https://doi.org/10.1145/3704922
18. Goodfellow I, Bengio Y, Courville A (2016) Deep Learning. MIT press, Cambridge
19. Loshchilov I, Hutter F (2017) Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101
20. Zhou P, Xie X, Lin Z (2024) Towards understanding convergence and generalization of AdamW. IEEE Trans Pattern Anal Mach Intell. https://doi.org/10.1109/tpami.2024.3382294
21. Bifet A, Holmes G, Pfahringer B (2010) Massive online analysis, a framework for stream classification and clustering. In: Proceedings of the first workshop on applications of pattern analysis, pp 44–50. https://doi.org/10.1145/1964897.1964906
22. Guo H, Zhang Y, Wang W (2024) Two-stage adaptive ensemble learning method for different types of concept drift. J Comput Res Dev 61(7):1799–1811. https://doi.org/10.7544/issn1000-1239.202330452
23. Zhu Q, Hu X, Zhang Y (2010) A double-window-based classification algorithm for concept drifting data streams. In: 2010 IEEE international conference on granular computing, pp 639–644. https://doi.org/10.1109/GrC.2010.125
24. Gomes HM, Bifet A, Read J, Barddal JP, Enembreck F, Pfharinger B, Holmes G, Abdessalem T (2017) Adaptive random forests for evolving data stream classification. Mach Learn 106(9):1469–1495. https://doi.org/10.1007/s10994-017-5642-8
25. Gomes HM, Read J, Bifet A (2019) Streaming random patches for evolving data stream classification. In: 2019 IEEE international conference on data mining (ICDM), pp 240–249. https://doi.org/10.1109/ICDM.2019.00034
26. Sahoo D, Pham Q, Lu J (2018) Online deep learning: Learning deep neural networks on the fly. In: Proceedings of the 27th international joint conference on artificial intelligence, pp 2660–2666. https://doi.org/10.48550/arXiv.1711.03705
27. Huang G, Liu Z, Pleiss G, Maaten L, Weinberger KQ (2022) Convolutional networks with dense connectivity. IEEE Trans Pattern Anal Mach Intell 44(12):8704–8716. https://doi.org/10.1109/TPAMI.2019.2918284
28. Pereira DG, Afonso A, Medeiros FM (2015) Overview of friedman's test and post-hoc analysis. Commun Stat-Simul Comput 44(10):2636–2653. https://doi.org/10.1080/03610918.2014.931971
29. Demšar J (2006) Statistical comparisons of classifiers over multiple data sets. J Mach Learn Res 7:1–30. https://doi.org/10.1007/s10846-005-9016-2

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.